Deep Reinforcement Learning

Bonus Unit 5: Imitation Learning in Godot

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction: **1**
- 1.1 Objectives: . . . 1
- 1.2 Prerequisites and requirements: . . . 1

2 The environment **2**

3 Getting started: **2**
- 3.1 Open the starter project in Godot . . . 2
- 3.2 Open the robot scene . . . 3
- 3.3 Click on the scroll next to AIController3D to open the script for editing . . . 3
- 3.4 Replace the `get_obs()` and `get_reward()` methods with the implementation below: . . . 3
- 3.5 Replace the `_physics_process()` and `reset()` methods with the implementation below: . . . 4
- 3.6 **Replace the `get_action_space()`, `get_action()`, and `set_action()` methods with the implementation below:** . . . 5
- 3.7 Replace the `_input` method with the implementation below: . . . 6
- 3.8 Open the demo record scene, and click on AIController3D node . . . 6
- 3.9 Let's record some demos: . . . 7
- 3.10 Export the game for training: . . . 8

4 Train our robot **8**
- 4.1 Download a copy of the imitation learning script from the Godot RL Repository. . . . 8
- 4.2 Run training using the arguments below: . . . 8
- 4.3 Let's test the trained agent . . . 9
- 4.4 Open the onnx inference scene . . . 9

5 (Optional) How to customize the environment **10**

6 Conclusion: **12**

1 Introduction:



Figure 1: Imitation Learning with Godot RL Agents: unit overview [original image]

Welcome to this bonus unit, where you will **train a robot agent to complete a mini-game level using imitation learning.**

At the end of the unit, **you will have a trained agent capable of solving the level as in the video**:

Video: Trained robot completing the level using ONNX inference
Watch the original video

1.1 Objectives:

- Learn how to use imitation learning with Godot RL Agents by training an agent to complete a mini-game environment using human-recorded expert demonstrations.

1.2 Prerequisites and requirements:

- It is recommended that you complete the previous chapter (Godot RL Agents) before starting this tutorial,
- Some familiarity with Godot is recommended, although completing the tutorial does not require any gdscript coding knowledge,
- Godot with .NET support (tested to work with 4.3.dev5 .NET, may work with newer versions too),
- Godot RL Agents (you can use `pip install godot-rl` in the venv/conda env),
- Imitation library,
- Time: ~1-2 hours to complete the project and training. It can be outside of this range depending on the hardware used.

2 The environment

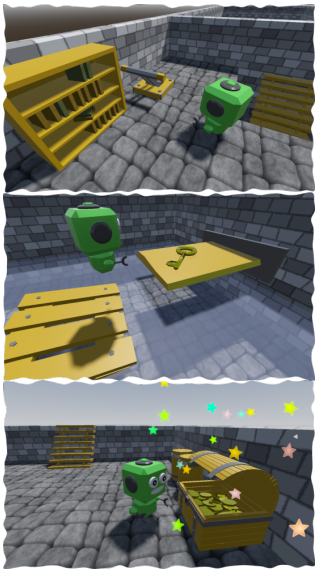


Figure 2: Mini-game level: lever, key, and treasure chest [original image]

The tutorial environment features a robot that needs to:

- Pull a lever to raise the stairs leading to the second room,
- Navigate to the key `[key]` and collect it while avoiding falling down into traps, water, or outside the map,
- Navigate back to the treasure chest in the first room, and open it. Victory! `[trophy]`

3 Getting started:

To get started, download the project from here (click on the download icon next to `GDRL-IL-Project.zip`). The zip file features both the "Starter" and "Complete" projects.

The game code is already implemented in the starter project and the nodes are configured. We will focus on:

- Implementing the code for the AIController node,
- Recording expert demonstrations,
- Training the agent and exporting an .onnx file which we can use for inference in Godot.

3.1 Open the starter project in Godot

Extract the zip file, open Godot, click "Import" and navigate to the `Starter\Godot` folder of the extracted archive.

3.2 Open the robot scene

> **Tip.** You can search for "robot" in the FileSystem search.

This scene contains a couple of different nodes, including the `robot` node, which contains the visual shape of the robot, `CameraXRotation` node which is used to rotate the camera "up-down" using the mouse in human control modes. The AI agent does not control this node since it is not necessary for learning the task. `RaycastSensors` node contains two Raycast sensors that help the agent to "sense" parts of the game world, including walls, floors, etc.

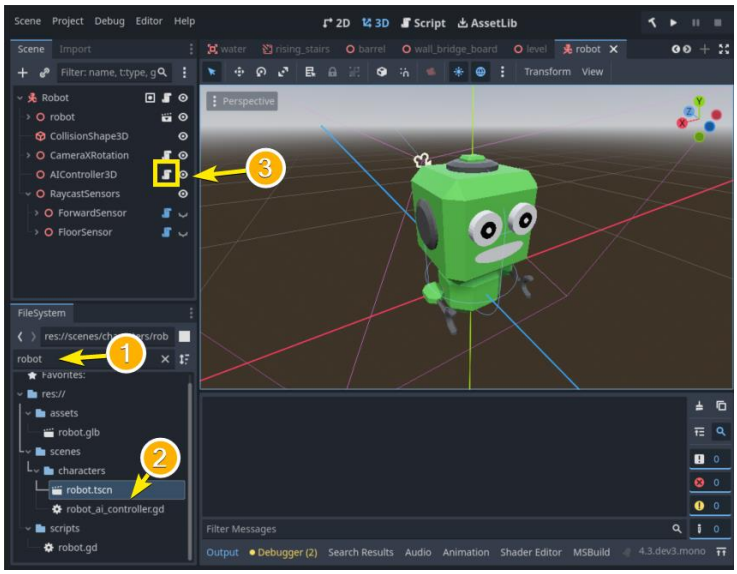


Figure 3: Godot editor: opening the robot scene [original image]

3.3 Click on the scroll next to AIController3D to open the script for editing

Tip. You might have to collapse the "robot" branch to find it more easily, or you can type `aicontroller` in the Filter box above the `Robot` node.

3.4 Replace the `get_obs()` and `get_reward()` methods with the implementation below:

```
func get_obs() -> Dictionary:
    var observations: Array[float] = []
    for raycast_sensor in raycast_sensors:
        observations.append_array(raycast_sensor.get_observation())

    var level_size = 16.0

    var chest_local = to_local(chest.global_position)
    var chest_direction = chest_local.normalized()
    var chest_distance = clampf(chest_local.length(), 0.0, level_size)

    var lever_local = to_local(lever.global_position)
    var lever_direction = lever_local.normalized()
    var lever_distance = clampf(lever_local.length(), 0.0, level_size)

    var key_local = to_local(key.global_position)
    var key_direction = key_local.normalized()
    var key_distance = clampf(key_local.length(), 0.0, level_size)

    var raft_local = to_local(raft.global_position)
    var raft_direction = raft_local.normalized()
    var raft_distance = clampf(raft_local.length(), 0.0, level_size)

    var player_speed = player.global_basis.inverse() * player.velocity.limit_length(5.0) / 5.0

    (
        observations
        .append_array(
            [
                chest_direction.x,
                chest_direction.y,
                chest_direction.z,
                chest_distance,
```

```
                lever_direction.x,
                lever_direction.y,
                lever_direction.z,
                lever_distance,
                key_direction.x,
                key_direction.y,
                key_direction.z,
                key_distance,
                raft_direction.x,
                raft_direction.y,
                raft_direction.z,
                raft_distance,
                raft.movement_direction_multiplier,
                float(player._is_lever_pulled),
                float(player._is_chest_opened),
                float(player._is_key_collected),
                float(player.is_on_floor()),
                player_speed.x,
                player_speed.y,
                player_speed.z,
            ]
        )
    )
    return {"obs": observations}

func get_reward() -> float:
    return reward
```

In `get_obs()`, we first get the obs from the two Raycast sensors added to the `AIController3D` node in the inspector, and add them to the obs, then we get the relative position vectors to chest, lever, key, and raft, which we separate into directions and distances, and then we add them to the obs as well.

We also add other game state info to the obs:

- has the lever has been pulled,
- was the key collected,
- was the chest opened,
- is the player on floor (also determines whether the player can jump),
- the normalized local velocity of the player.

We convert boolean values such as `_is_lever_pulled` to floats (0 or 1).

In `get_reward()`, we only need to return the current reward.

3.5 Replace the `_physics_process()` and `reset()` methods with the implementation below:

```
func _physics_process(delta: float) -> void:
    # Reset on timeout, this is implemented in parent class to set needs_reset to true,
    # we are re-implementing here to call player.game_over() that handles the game reset.
    n_steps += 1
    if n_steps > reset_after:
        player.game_over()

    # In training or onnx inference modes, this method will be called by sync node with actions
    provided,
    # For expert demo recording mode, it will be called without any actions (as we set the actions
    based on human input),
    # For human control mode the method will not be called, so we call it here without any actions
    provided.
    if control_mode == ControlModes.HUMAN:
        set_action()
```

```
    # Reset the game faster if the lever is not pulled.
    steps_without_lever_pulled += 1
    if steps_without_lever_pulled > 200 and (not player._is_lever_pulled):
        player.game_over()

func reset():
    super.reset()
    steps_without_lever_pulled = 0
```

3.6 Replace the get_action_space(), get_action(), and set_action() methods with the implementation below:

```
# Defines the actions for the AI agent ("size": 2 means 2 floats for this action)
func get_action_space() -> Dictionary:
    return {
        "movement": {"size": 2, "action_type": "continuous"},
        "rotation": {"size": 1, "action_type": "continuous"},
        "jump": {"size": 1, "action_type": "continuous"},
        "use_action": {"size": 1, "action_type": "continuous"}
    }

# We return the action values in the same order as defined in get_action_space() (important), but
    all in one array
# For actions of size 1, we return 1 float in the array, for size 2, 2 floats in the array, etc.
# set_action is called just before get_action by the sync node, so we can read the newly set values
func get_action():
    return [
        # "movement" action values
        player.requested_movement.x,
        player.requested_movement.y,
        # "rotation" action value
        player.requested_rotation.x,
        # "jump" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.jump_requested),
        # "use_action" action value (-1 if not requested, 1 if requested)
        -1.0 + 2.0 * float(player.use_action_requested)
    ]

# Here we set human control and AI control actions to the robot
func set_action(action = null) -> void:
    # If there's no action provided, it means that AI is not controlling the robot (human control),
    if not action:
        # Only rotate if the mouse has moved since the last set_action call
        if previous_mouse_movement == mouse_movement:
            mouse_movement = Vector2.ZERO

        player.requested_movement = Input.get_vector(
            "move_left", "move_right", "move_forward", "move_back"
        )
        player.requested_rotation = mouse_movement

        var use_action = Input.is_action_pressed("requested_action")
        var jump = Input.is_action_pressed("requested_jump")

        player.use_action_requested = use_action
        player.jump_requested = jump

        previous_mouse_movement = mouse_movement
    else:
        # If there is action provided, we set the actions received from the AI agent
        player.requested_movement = Vector2(action.movement[0], action.movement[1])
```

```
        # The agent only rotates the robot along the Y axis, no need to rotate the camera along X
    axis
        player.requested_rotation = Vector2(action.rotation[0], 0.0)
        player.jump_requested = bool(action.jump[0] > 0)
        player.use_action_requested = bool(action.use_action[0] > 0)
```

For `get_action()` (only needed if using the demo record mode), we need to provide the actions that we want the agent to send when it encounters the same state. It is important for the values to be in the correct range (`-1.0 to 1.0`), which is why we have the `-1 + 2 * variable` for boolean states, and in the correct order, as defined in `get_action_space()`.

In demo record mode, `set_action()` is called without providing actions, as we need to set the action values based on human input. In training/inference modes, the method is called with an `action` argument containing values for all of the actions provided by the RL model, so we have an `if/else` to handle both cases.

More info is included in the code comments.

3.7 Replace the `_input` method with the implementation below:

```
# Record mouse movement for human and demo_record modes
# We don't directly rotate in input to allow for frame skipping (action_repeat setting) which
# will also be applied to the AI agent in training/inference modes.
func _input(event):
    if not (heuristic == "human" or heuristic == "demo_record"):
        return

    if event is InputEventMouseMotion:
        var movement_scale: float = 0.005
        mouse_movement.y = clampf(event.relative.y * movement_scale, -1.0, 1.0)
        mouse_movement.x = clampf(event.relative.x * movement_scale, -1.0, 1.0)
```

This code part records mouse movement in case of human control and demo record modes.

Finally, save the script. We are ready for the next step.

3.8 Open the demo record scene, and click on AIController3D node

> **Tip.** You can search for “demo” in the FileSystem search, and you can search for “aicontroller” in the scene’s filter box.

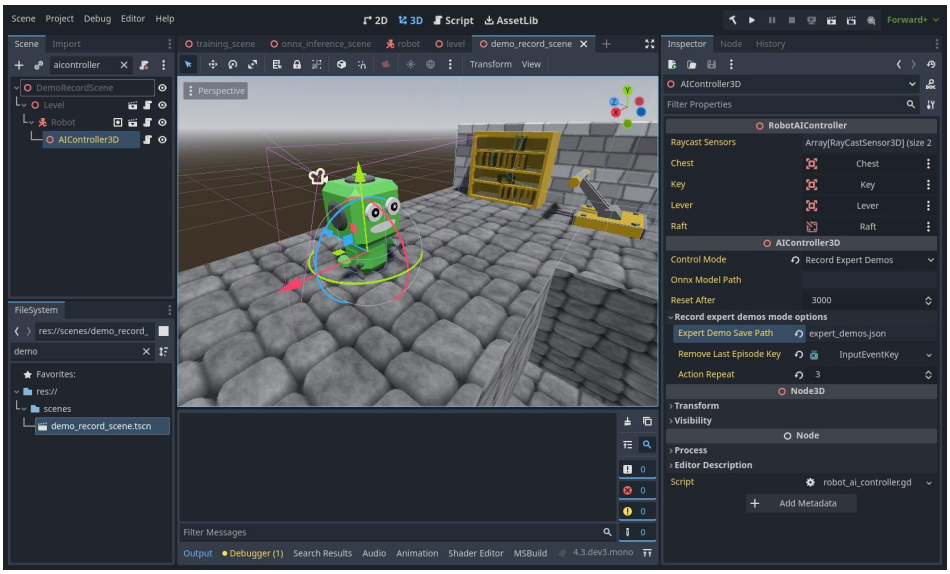


Figure 4: Godot editor: expert demonstration recording scene [original image]

You don't need to make any changes as everything is preset, but let's go over the things you would need to set in your own env:

The scene contains modified `Level > Robot > AIController3D` node settings:

- `Control Mode` is set to `Record Expert Demos`
- `Expert Demo Save Path` is filled out
- `Action Repeat` is set to the same value as is set for the `Sync` node in `training_scene` and `onnx_inference_scene`. This means that every action set by the agent is repeated for 3 physics frames. The setting in `AIController` adds the same action repeat to the human input (which introduces some lag) to match the same behavior. This is a fairly low value which doesn't introduce much lag. If you change this value, make sure to change it in all 3 places.
- `Remove Last Episode` key allows us to set a key that can be used to remove a failed episode during recording, without having to restart the entire session. E.g. if the robot falls in the water and the game resets, we can use this key to remove the previously recorded episode while recording the next one. It is set to `R`, but you can change it to any key by clicking on it, and then clicking on the `Configure` button.

Another way to make episode recording easier in challenging environments is to slow down the environment during recording. This can easily be done by clicking on the `Sync` node in the scene, and adjusting the `Speed Up` property (set to 1 by default).

3.9 Let's record some demos:

Tip. Note that the demos will only be saved if we have recorded at least one complete episode and closed the game window by clicking on "X" or pressing ALT+F4. Using the stop button in Godot editor will not save the demos. It's best to try recording just one episode first, then check if you see "expert_demos.json" in the filesystem or in the Godot project folder.

Make sure that you are still in the `demo_record_scene`, `press F6` and the demo recording will start.

Controls:

- mouse controls the camera (if you need to adjust mouse sensitivity, open the `robot` scene, click on the `Robot` node and adjust the `Rotation Speed`, keep it the same value for recording demos, training and inference),
- `WASD` controls the player movement,

- SPACE jumps,
- E activates the lever and opens the chest

You can take a few practice first to get familiar with the env. If you wish to skip recording demos, you can also find the pre-recorded demos in the completed project and use the `expert_demos.json` file from there.

The recorded demos should include at least 22-24 complete successful episodes. Multiple demo files can also be used in the training stage, so you don't have to record all demos in one go (you can change the file name using the `Expert Demo Save Path` property mentioned before).

Recording 23 episodes took me ~10 minutes (as the key has 2 alternating spawning positions, 22 or 24 would provide an equal distribution of key positions in the demos, but it is fairly close). When approaching the lever or chest, I pressed and held the E key slightly longer to ensure the action is recorded for multiple steps when near those objects. I also removed a couple of episodes that I didn't complete successfully by pressing the R key during the following episode.

Here's a sped-up video of the demo recording process:

Video: Recorded human expert demonstrations (sped-up video)
Watch the original video

3.10 Export the game for training:

You can export the game from Godot using `Project > Export`.

4 Train our robot

Tip. In order to start training, we'll first need to install the imitation library in the same venv / conda env where you installed Godot RL Agents by using: `pip install imitation`

4.1 Download a copy of the imitation learning script from the Godot RL Repository.

4.2 Run training using the arguments below:

```
sb3_imitation.py --env_path="path_to_ILTutorial_executable" --bc_epochs=100 --gail_timesteps
    =1450000 --demo_files "path_to_expert_demos.json" --n_parallel=4 --speedup=20 --
    onnx_export_path=model.onnx --experiment_name=ILTutorial
```

Set the env path to the exported game, and demo files path to the recorded demos. If you have multiple demo files add them with a space in between, e.g. `--demo_files demos.json demos2.json`.

You can also set a large amount of timesteps for `--gail_timesteps` and then manually stop training with `CTRL+C`. I used this method to stop training when the reward started to approach 3, which was at `total_timesteps | 1.38e+06`. That took ~41 minutes, and the BC pre-training took ~5.5 minutes on my PC using CPU for training.

To observe the environment while training, add the `--viz` argument. For the duration of the BC training, the env will be frozen as this stage doesn't use the env except to get some information about the observation and action spaces. During the GAIL training stage, the env rendering will update.

Here are the `ep_rew_mean` and `ep_rew_wrapped_mean` stats from the logs displayed using tensorboard, we can see that they are closely matching in this case:

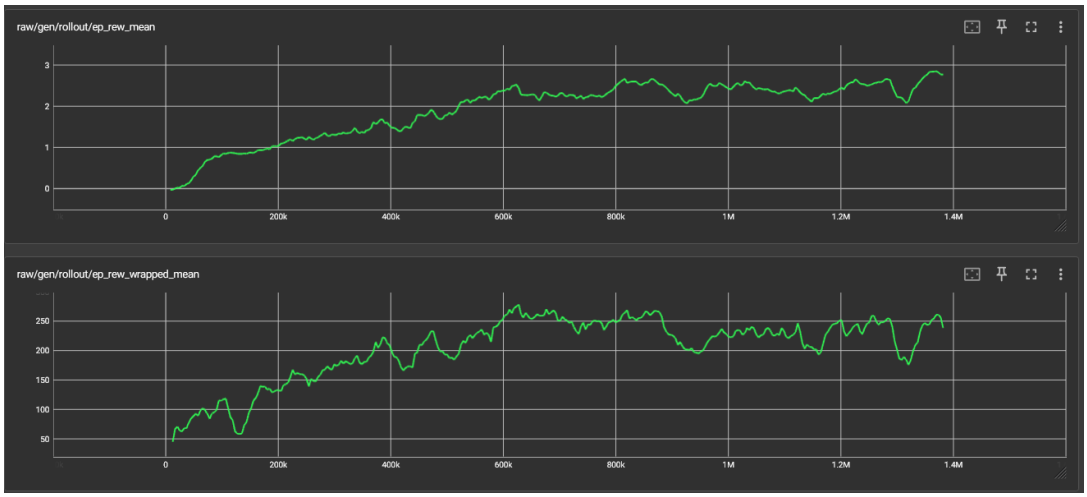


Figure 5: Training curves: ep_rew_mean and ep_rew_wrapped_mean in TensorBoard [original image]

> **Tip.** You can find the logs in `logs/ILTutorial` relative to the path you started training from. If making multiple runs, change the `--experiment_name` argument between each.

Even though setting the env rewards is not necessary and not used for the training here, a simple sparse reward was implemented to track success. Falling outside the map, in water, or traps sets `reward += -1`, while activating the lever, collecting the key, and opening the chest each set `reward += 1`. If the `ep_rew_mean` approaches 3, we are getting a good result. `ep_rew_wrapped_mean` is the reward from the GAIL discriminator, which does not directly tell us how successful the agent is at solving the environment.

4.3 Let's test the trained agent

After training, you'll find a `model.onnx` file in the folder you started the training script from (you can also find the full path to the `.onnx` file in the training log in the console, near the end). **Copy it to the Godot game project folder.**

4.4 Open the onnx inference scene

This scene, like the demo record scene, uses only one copy of the level. It also has it's `Sync` node mode set to `Onnx Inference`.

Click on the `Sync` node and set the `Onnx Model Path` property to `model.onnx`.

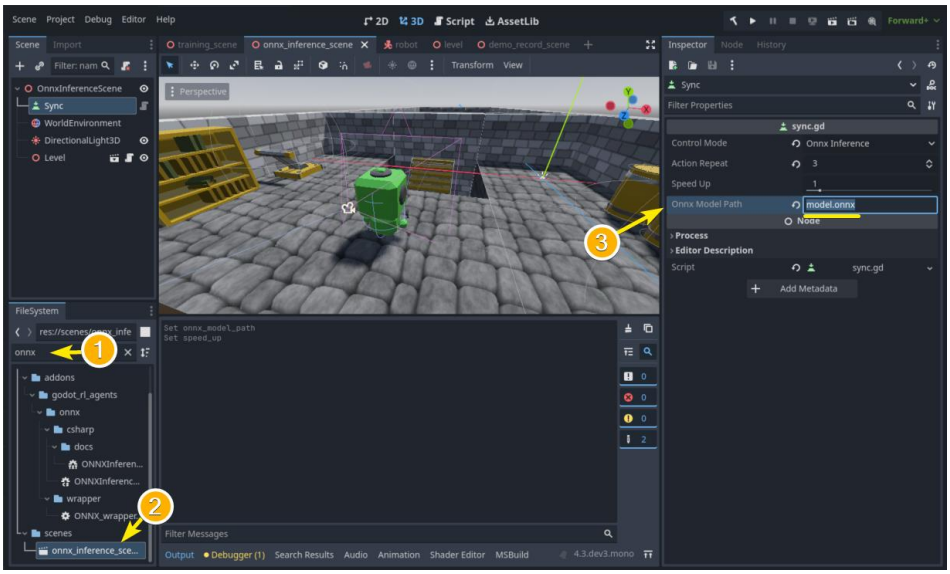


Figure 6: Godot editor: configuring the ONNX inference scene [original image]

Press F6 to start the scene and let's see what the agent has learned!

Video of the trained agent:

> **Video:** Trained robot completing the level using ONNX inference
> Watch the original video

It seems the agent is capable of collecting the key from both positions (left platform or right platform) and replicates the recorded behavior well. **If you're getting similar results, well done, you've successfully completed this tutorial!** `[trophy][applause]`

If your results are significantly different, note that the amount and quality of recorded demos can affect the results, and adjusting the number of steps for BC/GAIL stages as well as modifying the hyper-parameters in the Python script can potentially help. There's also some run-to-run variation, so sometimes the results can be slightly different even with the same settings.

5 (Optional) How to customize the environment

If you'd like to customize the game level, open the level scene `res://scenes/level.tscn`, then open the `res://scenes/modules/` folder in the Godot FileSystem:

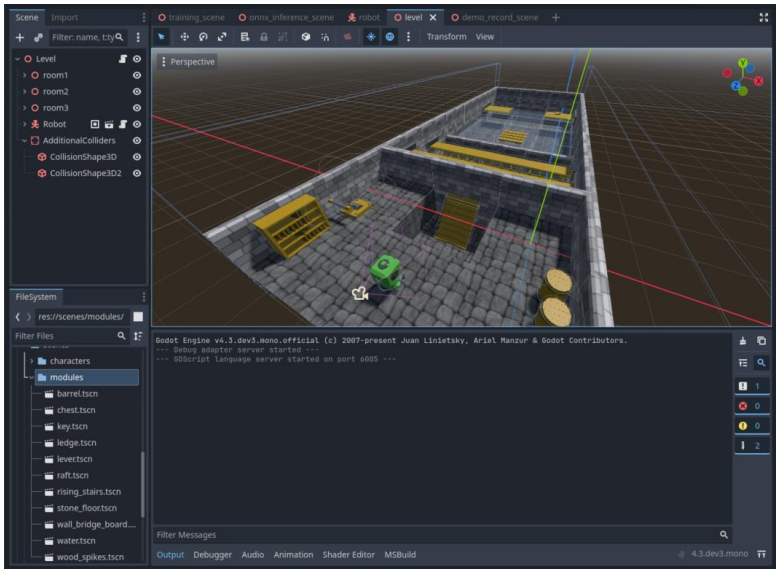


Figure 7: Level scene and reusable room modules in Godot [original image]

The level contains 3 rooms made using the modules, robot, and some additional colliders which prevent the ability to complete the level by climbing on a wall in the first room and reaching the key that way. By adding the modules to the scene, you can add new rooms and items.

If you click on the Key node (it's in `Room3`, you can also search for it), then click on `Node > Signals`, you will see that the `collected` signal is connected to both the robot and the chest. We use this to track whether the robot has collected the key, and to unlock the chest. The same system is applied for using the lever to activate the stairs, and if you add more levers/stairs/keys, you can connect them using signals.

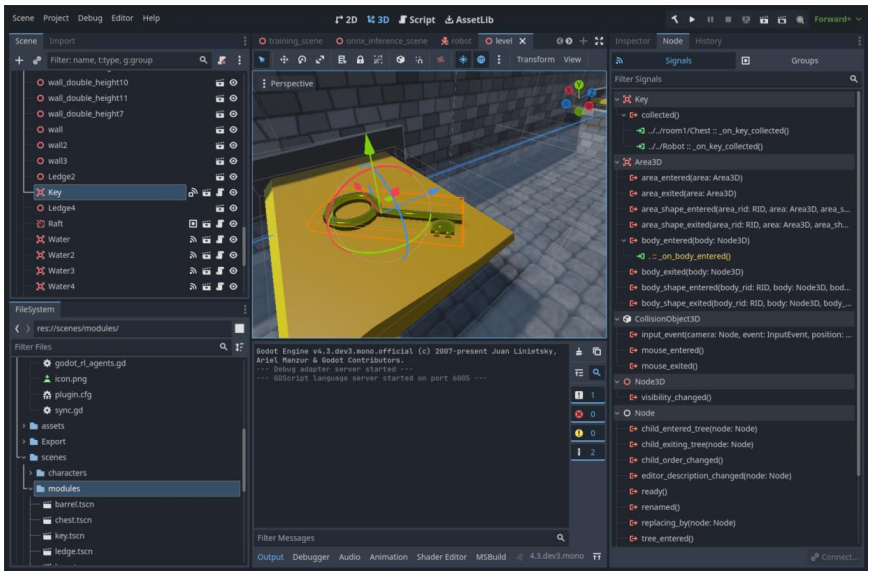


Figure 8: Godot signals: connecting collectible objects and level interactions [original image]

If you switch to `Groups`, you will see that the key is a member of the `resetable` group. In the same group we have the raft, lever, chest, player, and can add any node that needs to be reset when the episode resets.

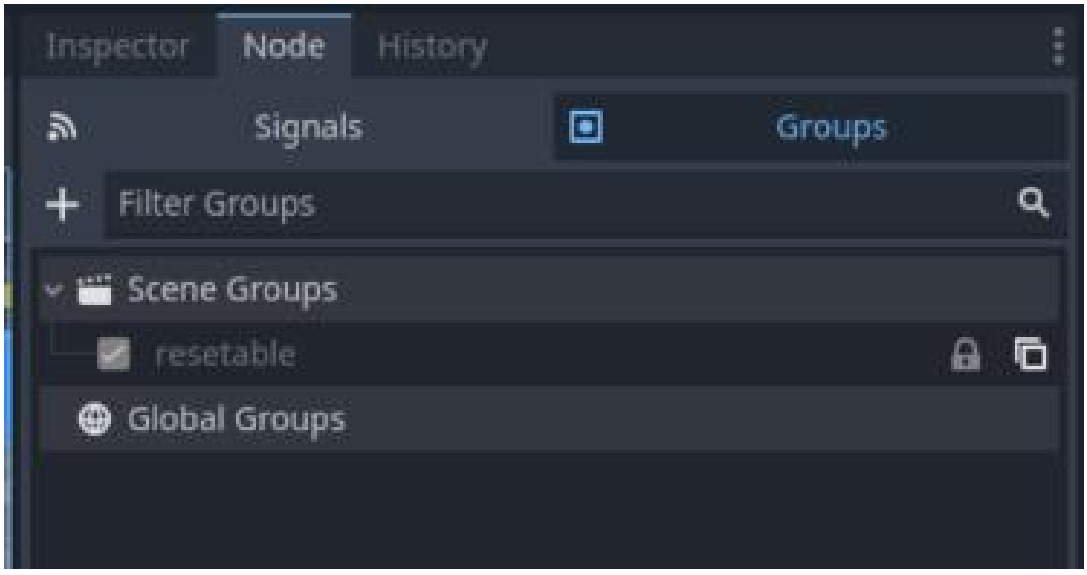


Figure 9: Nodes in the resetable group for episode resets [original image]

For this to work, every object that is in the `resetable` group also needs to implement the `reset()` method, which takes care of resetting that object.

Because we have multiple instances of the level scene for training, we don't reset all `resetables`, but only those within the same scene. In `level_manager.gd`, we have a method `reset_all_resetables()` that takes care of this, and it is called by the robot script when resetting is needed.

After changing the level size, updating the `level_size` variable in `robot_ai_controller.gd` is also needed. For this, just roughly measure the longest dimension of the level, and update the variable.

If you change the amount of objects that need to be tracked by the `AIController` (levers, rafts, etc.), you will need to update the relevant code in the script, include export properties for those objects, then connect them in the inspector properties of `AIController` in the level scene:

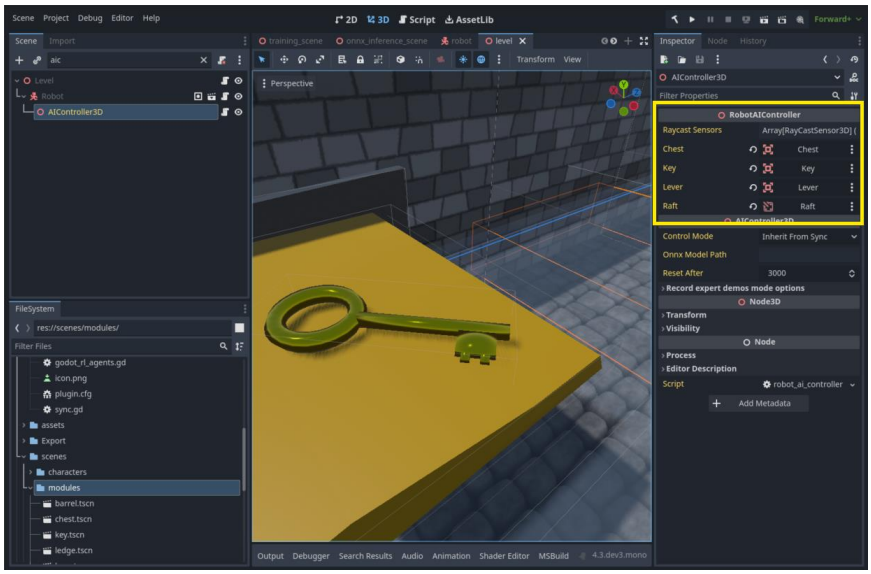


Figure 10: Godot inspector: AIController object references [original image]

After this, you may also need to update the same properties of the `AIController` in the demo record scene as well.

6 Conclusion:

Congratulations on finishing this bonus unit! You have learned the process of recording expert demonstrations and training the agent using IL, which can be an alternative to training in-game agents with RL in some cases.

This tutorial was written by Ivan Dodic. Thanks to Edward Beeching and Thomas Simonini for their reviews and feedback.